

19

Attention

The breakthrough idea: focus on what matters

The bridge from RNNs to Transformers and modern AI

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A way for a model to focus on the important parts
The problem it fixed	RNNs forget and squeeze everything into one vector
Big idea	Look at ALL words, weigh which ones matter
Led to	Transformers, BERT, GPT — all modern AI
Famous paper	'Attention Is All You Need' (2017)

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	The Problem with RNNs	Why we needed a new idea
2	What is Attention?	Focusing on what matters
3	A Human Analogy	How you already use attention
4	Query, Key, Value	The three roles, made simple
5	How Attention Works	Step by step
6	Self-Attention	Words looking at each other
7	Reading an Attention Map	Seeing what the model focuses on
8	Multi-Head Attention	Many viewpoints at once
9	Why It Changed Everything	The road to Transformers
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. The Problem with RNNs

LSTMs and GRUs were good, but they had a deep weakness. They read a sentence one word at a time and squeeze the whole meaning into a single small memory vector. For a long sentence, that one vector simply can't hold everything. This is called the **bottleneck problem** (■■■■■■■ — ■■■■■■■■ ■■).

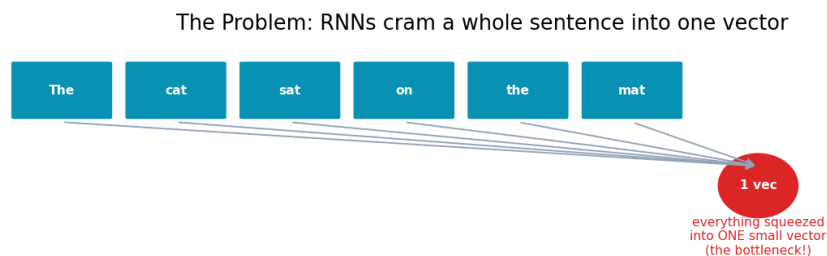


Fig 1 — An RNN crams an entire sentence into one small vector. Long sentences don't fit.

There was a second problem too: RNNs read strictly in order, one word after another. They can't be sped up by doing words in parallel, so they're slow to train on big data. We needed something that could look at all words at once AND focus on the important ones. That something is **attention**.

■ *This is the most important topic in the deep learning section. Attention is the idea that powers every modern AI model — GPT, BERT, and the model writing this. Take your time here.*

2. What is Attention?

Attention (■■■■■■■■■■ — ■■■■■■■■) lets a model look at ALL the words at once and decide which ones to focus on for each task. Instead of squeezing everything into one vector, it keeps every word available and learns how much to 'pay attention' to each one.

Take the sentence: 'The animal did not cross the street because it was tired.' What does 'it' refer to — the animal or the street? To figure this out, the model should focus mostly on 'animal'. Attention lets it do exactly that: when processing 'it', it puts most of its focus on 'animal'.

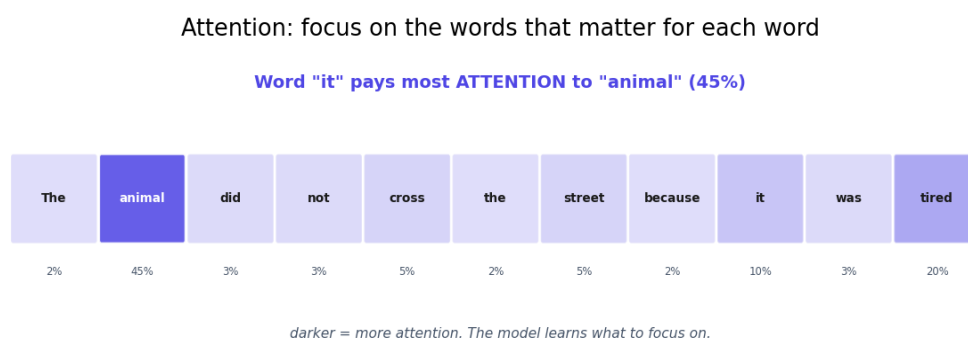


Fig 2 — When understanding 'it', the model pays the most attention to 'animal'. Darker = more focus.

So attention produces a set of **weights** — numbers saying how important each word is for the current step. High weight = pay lots of attention; low weight = mostly ignore. These weights are learned from data, not hand-coded.

3. A Human Analogy

You use attention every day. When you read a page looking for one fact, your eyes skim past most words and lock onto the relevant ones. When you listen at a noisy party, you tune out the crowd and focus on your friend's voice. You don't process everything equally — you focus.

Attention in a neural network is the same idea. For each word it processes, the model asks: 'which other words help me understand this one?' Then it focuses on those and downplays the rest. This simple ability turns out to be incredibly powerful.

■ *The key shift: RNNs had ONE fading memory. Attention gives the model direct access to EVERY word, all the time, with the freedom to focus wherever it needs. Nothing gets forgotten.*

4. Query, Key, Value — The Three Roles

Attention works like a smart search. Three roles make it happen, and they have an easy real-world parallel: searching for a video online.

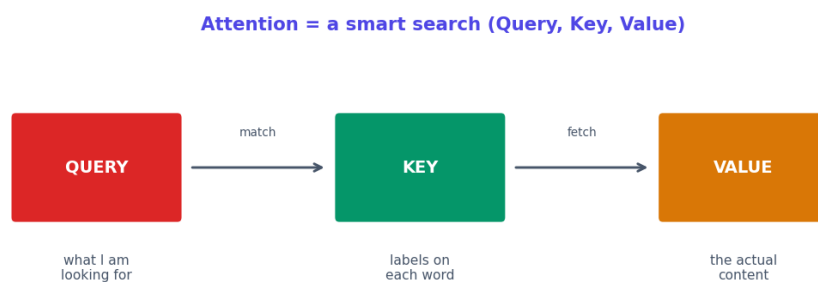


Fig 3 — Query (what you want), Key (labels to match against), Value (the actual content you get back).

Role	What it is	Video search analogy
Query	What the current word is looking for	Your search text
Key	A label on each other word	Each video's title/tags
Value	The actual content of each word	The video you watch

Here's the flow: the **Query** (what I'm looking for) is compared against every **Key** (the labels). Where the query and a key match well, that word gets a high attention weight. Then the model collects the **Values** of the words, weighted by how well they matched. The result is a focused summary built from the most relevant words.

5. How Attention Works — Step by Step

- **Step 1:** For the current word, create its Query. For every word, create a Key and a Value.
- **Step 2:** Compare the Query against every Key to get a match score for each word.
- **Step 3:** Turn those scores into weights that add up to 1 (using softmax). High score → high weight.
- **Step 4:** Multiply each word's Value by its weight, and add them all up.
- **Step 5:** The result is the attention output for the current word — a blend focused on the words that mattered most.

The whole thing is captured in one famous formula. You don't need to compute it by hand, but it's nice to recognise:

$$\text{Attention} = \text{softmax}(Q \times K^T / \sqrt{d}) \times V$$

In words: match queries to keys ($Q \times K$), scale them, turn into weights (softmax), then use those weights to blend the values ($\times V$). That single formula is the engine of modern AI.

6. Self-Attention — Words Looking at Each Other

The most powerful form is **self-attention** (■■■■■-■■■■■■■■■■). Here, every word in a sentence looks at every other word in the same sentence — including itself — to build a richer understanding of each one based on its context.

This is how the model resolves 'it = animal'. Every word gathers context from all the others. 'It' looks around, sees 'animal', and links to it. 'Cross' looks at 'street'. Each word becomes context-aware. Because all words do this at the same time, self-attention can be computed in parallel — which is why it's so much faster than an RNN.

■ *Self-attention solved both RNN problems at once: no bottleneck (every word stays available) and full parallelism (all words processed together, not one by one). This is the breakthrough.*

7. Reading an Attention Map

One lovely thing about attention: you can SEE what the model focuses on. An **attention map** is a grid showing how much each output word looked at each input word. Bright squares mean strong attention.

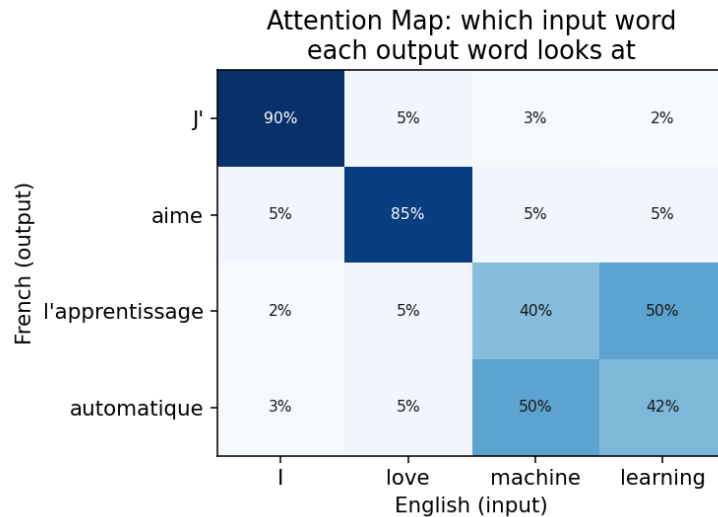


Fig 4 — A translation attention map. Each French word focuses on the English word(s) it came from.

In this English-to-French translation, you can see 'J'aime' lines up with 'I love', and the French words for 'machine learning' attend to the English 'machine' and 'learning'. The model learned these alignments on its own. This makes attention models more interpretable than older black-box networks — you can actually inspect what they focused on.

8. Multi-Head Attention — Many Viewpoints

Instead of paying attention just one way, models use **multi-head attention** (■■■■■■-■■■) — several attention 'heads' running at the same time, each focusing on different things.

One head might track grammar (which noun goes with which verb). Another might track meaning (which words are about the same topic). Another might track position. Each head learns its own viewpoint, and the model combines them all. It's like having several experts each reading the sentence with a different focus, then pooling their insights.

```
import torch.nn as nn

# PyTorch has attention built in
attention = nn.MultiheadAttention(
    embed_dim=512,    # size of each word vector
    num_heads=8,      # 8 different attention viewpoints
    batch_first=True)

# Self-attention: query, key, value are all the same input
output, weights = attention(x, x, x)
# 'weights' is the attention map you can visualise
```


9. Why It Changed Everything

In 2017, a famous paper titled 'Attention Is All You Need' showed something surprising: you could throw away the RNN entirely and build a model from attention alone. This new design was the **Transformer** — the next topic.

Because attention processes all words in parallel and never forgets, Transformers could be trained on enormous amounts of text, very fast, on many GPUs at once. This unlocked the giant models we have today.

Model	Built on attention, used for
BERT	Understanding text (search, classification)
GPT	Generating text (chatbots, writing)
Vision Transformer	Applying attention to images
Modern LLMs	Everything — chat, code, reasoning

Every one of these is built on the attention idea you just learned. Master this concept and the rest of modern AI becomes far easier to understand.

10. Common Mistakes

Misunderstanding	The reality
'Attention is one fixed rule'	It's learned from data, like other weights
'It replaces understanding'	It's a focusing tool, part of a bigger model
'Q, K, V are complicated'	Just: what I want, labels, content
'Self-attention needs order'	It looks at all words at once, in parallel
'One head is enough'	Multiple heads catch different patterns
'Only for text'	Works for images, audio, and more too

Quick Reference Cheatsheet

Concept	Meaning
Attention	focus on the words that matter most
Bottleneck problem	RNNs squeeze everything into one vector
Attention weights	how much focus each word gets (sum to 1)
Query (Q)	what the current word is looking for
Key (K)	a label on each word to match against

Value (V)	the actual content fetched
Softmax	turns match scores into weights
Self-attention	every word looks at every other word
Attention map	grid showing what focused on what
Multi-head	several attention views at once
In PyTorch	<code>nn.MultiheadAttention(embed_dim, num_heads)</code>
Led to	Transformers, BERT, GPT, modern LLMs

The one idea to remember: attention lets a model look at all words at once and focus on the ones that matter for each word — using Query, Key, and Value. No bottleneck, no forgetting, and fully parallel. This single idea powers all of modern AI.

Next Topic: Transformers — the architecture built entirely from attention. This is the engine inside GPT, BERT, and every large language model. Everything so far has been building to this.